

Programrendszerek fejlesztése projekt dokumentáció

Használt technológiák és motiváció

A projekt technológiai háttere

A projekt egy webalkalmazás, amely ASP.NET Core backend és Angular frontend segítségével készült. A rendszer célja egy fitness csoport foglalási rendszer megvalósítása, ahol a felhasználók megtekinthetik a fitness csoportokat, időpontokat, termeket, foglalásokat hozhatnak létre, valamint értékeléseket adhatnak le. Az alkalmazáson belül az adminisztrátorok megnézhetik egy *dashboard* oldalon a foglalásokat, az összértékeléseket, a regisztrált felhasználókat, valamint tudják kezelni a felhasználók jelentkezéseit és a hozzászólásait.

A választott technológiai stack:

- ASP.NET Core
- Angular
- Entity Framework Core
- SQL
- REST API

Ez a stack jól illeszkedik egy olyan rendszerhez, ahol fontos a strukturált adatkezelés, a több szerepkör kezelése, a biztonságos bejelentkezés, az adminisztrációs funkciók és a modern felhasználói felület.

ASP.NET Core

Az ASP.NET Core a projekt szerveroldali technológiája. A backend feladata, hogy biztosítsa az API végpontokat, kezelje az üzleti logikát, kommunikáljon az adatbázissal, és kiszolgálja az Angular frontend kéréseit. Ez a keretrendszer jó választás volt ehhez a projekthez, mert stabil, jól strukturálható és alkalmas komplexebb webalkalmazások fejlesztésére.

Egy fitness foglalási rendszerben több egymáshoz kapcsolódó adatot kell kezelni. Ilyenek például a felhasználók, szerepkörök, csoportok, edzők, időpontok, termek, foglalások és

értékelések. Az ASP.NET Core jól támogatja az ilyen típusú, rétegzett felépítésű alkalmazásokat.

A projektben az ASP.NET Core felel többek között:

- a bejelentkezésért és regisztrációért
- a szerepkörök kezeléséért
- az adatok kezeléséért és validálásáért
- az API válaszok előállításáért

A backendben elkülöníthető:

- a Controller réteg
- az adatkezelési réteg
- a modellek
- a DTO-k
- az üzleti logika
- az adatbázis-kapcsolat

Ez átláthatóbbá teszi a kódot, és hosszabb távon könnyebb karbantartást biztosít.

Angular

Az Angular a projekt frontend technológiája. Ez biztosítja a felhasználói felületet, amelyen keresztül a vendégek, tagok és adminisztrátorok használhatják a rendszert.

Az Angular feladatai:

- oldalak és komponensek megjelenítése
- felhasználói interakciók kezelése
- űrlapok kezelése
- validációk megjelenítése
- API kérések küldése a backend felé
- API válaszok feldolgozása
- foglalási és értékelési felületek kezelése
- adminisztrációs oldalak megjelenítése

Az Angular komponensalapú felépítése miatt a projekt külön részekre bontható. Például külön komponens készülhet a csoportok listázására, a csoport részleteire, a foglalások kezelésére, a bejelentkezésre, regisztrációra és az admin felületekre.

Az Angular előnye, hogy nagyobb webalkalmazásokhoz is jól használható. A fitness foglalási rendszer nem csak egy egyszerű statikus oldal, hanem több funkcióból álló alkalmazás.

Az Angular hasznos ebben a projektben, mert:

- komponensekre bontható a felület,
- jól kezelhetők az űrlapo
- támogatja az útvonalkezelést
- könnyen kommunikál REST API-val
- TypeScriptet használ
- átláthatóbb frontend struktúrát ad

A TypeScript használata külön előny, mert típusosabbá és biztonságosabbá teszi a frontend fejlesztést. Ez csökkenti a hibák esélyét, különösen akkor, amikor a frontend DTO-kat és API válaszokat kezel.

Entity Framework Core

Az Entity Framework Core az adatbázis-kezelést segíti. ORM-ként működik, vagyis lehetővé teszi, hogy az adatbázis tábláit C# osztályokon keresztül kezeljük. A projektben az Entity Framework Core kapcsolja össze az alkalmazás modelljeit az adatbázissal. Az Entity Framework Core azért hasznos, mert nem kell minden adatbázis-műveletet kézzel SQL-ben megírni. A fejlesztés C# objektumokkal történik, az EF Core pedig gondoskodik az adatbázis-műveletek végrehajtásáról. Ez különösen fontos egy fitness foglalási rendszerben, ahol sok kapcsolat van az entitások között. Például egy foglalás kapcsolódik egy felhasználóhoz és egy időponthoz, az időpont kapcsolódik egy csoporthoz és egy teremhez, a csoport pedig kapcsolódik egy edzőhöz.

Relációs adatbázis

A projekt relációs adatbázist használ. Ez megfelelő választás, mert a rendszerben sok egymáshoz kapcsolódó adat található.

A főbb adatok:

- felhasználók
- szerepkörök
- fitness csoportok
- edzők
- termek

- időpontok
- foglalások
- értékelések

Repository minta

A Repository minta előnye, hogy elkülöníti az adatbázis-műveleteket az alkalmazás többi részétől. Ez jobb projektstruktúrát ad, mert a Controller nem közvetlenül az adatbázissal dolgozik, hanem repository-n keresztül éri el az adatokat. Egy fitness foglalási rendszerben ez azért fontos, mert több entitással is hasonló műveleteket kell végezni. Például a felhasználók, csoportok, időpontok, foglalások és értékelések mind lekérdezhetők, módosíthatók vagy törölhetők.

Unit of Work minta

A Unit of Work minta célja, hogy több repository művelet egy közös adatbázis-kontextuson keresztül fusson. Ez azt jelenti, hogy több adatbázis-művelet egy logikai egységként kezelhető. A Unit of Work minta hasznos, mert segít megőrizni az adatok konzisztenciáját. Ha egy összetett művelet közben hiba történik, akkor a tranzakció visszavonható. Ez megakadályozza, hogy a rendszer félkész vagy hibás adatállapotba kerüljön. Ez a fitness foglalási rendszerben különösen fontos. Például nem lenne megfelelő, ha egy foglalás részben létrejönne, de a kapcsolódó adatok nem frissülnének megfelelően.

REST API alapú kommunikáció

Az ASP.NET Core backend és az Angular frontend REST API-n keresztül kommunikál egymással.

A frontend HTTP kéréseket küld a backendnek, például:

- felhasználó bejelentkezése,
- regisztráció,
- csoportok lekérése,
- időpontok lekérése,
- foglalás létrehozása,
- értékelés létrehozása,
- adminisztrációs adatok lekérése.

A backend feldolgozza ezeket a kéréseket, adatbázis-műveleteket hajt végre, majd választ küld vissza a frontendnek. A REST API alapú felépítés előnye, hogy a frontend és backend jól elkülönül egymástól.

Ez azt jelenti, hogy:

- a frontend külön fejleszthető
- a backend külön fejleszthető
- az API más klienssel is használható lenne
- a rendszer könnyebben bővíthető
- a projekt szerkezete tisztább

DTO-k használata

A projektben DTO-k, vagyis Data Transfer Object osztályok is használatban vannak.

A DTO-k célja, hogy az API ne közvetlenül az adatbázis-entitásokat küldje vissza, hanem csak azokat az adatokat, amelyekre a frontendnek szüksége van. A DTO-k használata biztonságosabbá és átláthatóbbá teszi az API kommunikációt.

Miért hasznos ez a stack a fitness témakörhöz?

A fitness foglalási rendszerben több szerepkör jelenik meg: vendég, felhasználó, edző, adminisztrátor. A választott technológiai stack jól támogatja ezt. Az ASP.NET Core backend képes kezelni a jogosultságokat, az Angular frontend pedig különböző felületet jeleníthet meg a felhasználó szerepköre alapján. Ez azért fontos, mert nem minden felhasználónak szabad ugyanazokat a funkciókat elérnie. Egy vendég csak megtekinti az órákat, egy tag foglalhat és értékelhet, az adminisztrátor pedig kezelheti a rendszer adatait.

A rendszernek ellenőriznie kell:

- melyik időpontra történik a foglalás
- ki foglal
- van-e még szabad hely
- milyen állapotban van a foglalás
- mikor jött létre a foglalás

A frontend külön oldalakon vagy komponensekben mutathatja:

- az összes csoportot
- egy csoport részleteit
- a hozzá tartozó időpontokat
- az elérhető helyeket
- az értékeléseket

Egy fitness foglalási rendszerben az adminisztrátornak kezelnie kell a rendszer adatait

- csoportok
- időpontok
- terme
- felhasználók
- foglalások
- értékelések

A vendégek számára lehetővé teszi:

- csoportok megtekintését,
- időpontok megtekintését,
- leírások elérését.
- regisztrációt

A tagok számára lehetővé teszi:

- bejelentkezést,
- foglalások létrehozását
- foglalások megtekintését
- értékelések leadását

Az adminisztrátorok számára lehetővé teszi:

- adatok kezelését
- csoportok listázását
- időpontok kezelését
- foglalások áttekintését
- felhasználók és szerepkörök kezelését
- összesített adatok megtekintését

Nem-funkcionális követelmények

Karbantarthatóság

A projekt több kisebb projektre van bontva:

- FitnessClassBookingWeb.DataAccess
- FitnessClassBookingWeb.Models
- fitnessclassbookingwebapp.client
- FitnessClassBookingWebApp.Server

Ez jó döntés, mert elkülönülnek a felelősségi körök. A modellek, adatkezelés, frontend és backend külön részekben kezelhetők.

Bővíthetőség

A rétegzett felépítés miatt később könnyebb új funkciókat hozzáadni. Például egy új Waitlist entitás vagy online fizetési funkció bevezetése nem igényelné az egész rendszer újraírását.

Biztonság

A bejelentkezés, szerepkörök, jelszó hash-elés és DTO-k használata hozzájárul a biztonságosabb működéshez.

Felhasználói élmény

Az Angular segítségével a rendszer modern és interaktív felületet tud biztosítani. A felhasználók gyorsan megtekinthetik a csoportokat, időpontokat, foglalásokat és értékeléseket.

Ez fitness témakörben különösen fontos, mert a felhasználó gyorsan szeretne dönteni arról, hogy melyik órára jelentkezik.

A DataAccess projekt

A FitnessClassBookingWeb.DataAccess projekt felelős az alkalmazás adatbázis-kezelési rétegéért. Ez a réteg választja el az üzleti logikát és az API működését az adatbázis közvetlen kezelésétől. Azokat az osztályokat tartalmazza, amelyek az adatbázis-kapcsolatot, a táblák lekérdezését, a kezdőadatokat betöltését, a migrációkat, valamint az általános adatkezelési műveleteket valósítják meg.

A projektben megtalálható főbb fájlok és mappák:

- ApplicationDbContext.cs
- DbInitializer.cs
- Migrations

- Repository.cs
- UnitOfWork.cs

ApplicationDbContext

Az ApplicationDbContext az Entity Framework Core központi adatbázis-kezelő osztálya. Ezen keresztül történik az alkalmazás adatbázisának kezelése.

A projektben ilyen táblák szerepelnek például:

- Users
- Roles
- UserRoles
- Groups
- Schedules
- Rooms
- Bookings
- Reviews

Ezek az entitások a fitness foglalási rendszer fő adatait tárolják. A Users tábla a felhasználókat, a Roles a szerepköröket, a Groups a fitness csoportokat, a Schedules az órarendi időpontokat, a Rooms a termeket, a Bookings a foglalásokat, a Reviews pedig az értékeléseket tartalmazza.

Egy felhasználó több foglalást is létrehozhat, egy csoporthoz több időpont tartozhat, egy időponthoz több foglalás kapcsolódhat, valamint egy felhasználó értékeléseket is adhat a csoportokhoz. A kapcsolatoknál több helyen külön törlési viselkedés is be van állítva. Ez megakadályozza, hogy egy kapcsolódó adat törlése nem kívánt adatvesztést okozzon. Ez különösen fontos a foglalások, értékelések, edzők és időpontok esetében.

DbInitializer

A DbInitializer feladata az adatbázis előkészítése és kezdőadatokkal való feltöltése. Az inicializáló osztály először lefuttatja az adatbázis migrációkat, majd ellenőrzi, hogy az adatbázis tartalmaz-e már adatokat. Ha az adatbázis még üres, akkor létrehozza a szükséges kezdőadatokat.

A rendszerben három alapvető szerepkör szerepel:

Admin

Coach

Member

Ez illeszkedik a projekt specifikációjához, mert a rendszernek külön kell kezelnie az adminisztrátori, edzői és tagi jogosultságokat. A felhasználók jelszavai nem egyszerű szöveggként kerülnek eltárolásra, hanem hash-elve. Ez biztonsági szempontból fontos, mert az adatbázisban nem jelenik meg közvetlenül az eredeti jelszó.

Migrációk

A Migrations mappa az Entity Framework Core migrációit tartalmazza. A migrációk írják le, hogyan kell az adatbázis szerkezetét létrehozni vagy módosítani. Például a Groups tábla kapcsolódik a Users táblához az edző azonosítóján keresztül. A Schedules tábla kapcsolódik a Groups és Rooms táblákhoz. A Bookings tábla kapcsolódik a Users és Schedules táblákhoz. A Reviews tábla pedig kapcsolódik a Users és Groups táblákhoz.

Repository minta

A projektben megvalósításra került a Repository tervezési minta. A Repository minta célja, hogy az adatbázis-műveletek ne közvetlenül a Controller vagy Service rétegben jelenjenek meg, hanem egy külön adatkezelő rétegen keresztül történjenek.

Ez tisztább és karbantarthatóbb szerkezetet eredményez. A repository réteg egységes módon kezeli az alapvető adatbázis-műveleteket, például a lekérdezést, létrehozást, módosítást és törlést.

Unit of Work minta

Célja, hogy több repository művelet egy közös adatbázis-kontextuson belül fusson, és a változások mentése központilag történjen. Ez azért hasznos, mert egy összetettebb művelet során több tábla is érintett lehet.

A Models projekt

A FitnessClassBookingWeb.Models projekt tartalmazza az alkalmazás adatmodelljeit és DTO osztályait. Ez a projekt határozza meg, hogy milyen adatszerkezetekkel dolgozik a rendszer.

A projekt két fő részből áll:

- Entitás modellek
- DTO osztályok

Az entitás modellek az adatbázis tábláit reprezentálják, míg a DTO-k az API-n keresztül történő adatküldéshez és adatfogadáshoz használatosak. Ez a szétválasztás azért hasznos, mert nem minden esetben célszerű a teljes adatbázis-entitást visszaküldeni a frontendnek. A DTO-k segítségével csak azok az adatok kerülnek átadásra, amelyekre az adott funkciónál valóban szükség van.

A projektben az alábbi fő entitások találhatók:

- User
- Role
- UserRole
- Group
- Schedule
- Room
- Booking
- Review

User modell

A User modell több kapcsolattal is rendelkezik:

- UserRoles
- Bookings
- Reviews
- CoachingGroups

Ez azt jelenti, hogy egy felhasználónak lehet több szerepköre, több foglalása, több értékelése, illetve edzőként több csoporthoz is kapcsolódhat.

Role modell

A rendszerben például ilyen szerepkörök használhatók:

- Admin
- Coach
- Member

A UserRoles kapcsolat segítségével egy szerepkör több felhasználóhoz is hozzárendelhető.

CreateReviewRequest

A `CreateReviewRequest` egy egyszerű kérésmodell, amely új értékelés létrehozásakor használható. A `Rating` kötelező, és 1 és 5 közötti értéket vehet fel. A `Comment` szintén kötelező. Ez a DTO azért hasznos, mert értékelés létrehozásakor nincs szükség minden review adatra. Például a `ReviewId`, `UserId`, `GroupId` és `CreatedAt` értékeket a rendszer is meghatározhatja a kérés feldolgozása közben.

DashboardStats

A `DashboardStats` a kezdőoldali vagy adminisztrátori statisztikai adatok tárolására szolgál. Megjeleníthető, hogy hány fitness csoport van a rendszerben, hány órarendi időpont érhető el, hány aktív felhasználó van, mennyi foglalás történt, és mennyi az átlagos értékelés.

Validáció a modellekben

A DTO osztályok több helyen használnak validációs attribútumokat.

Ilyenek például:

- `[Required]`
- `[EmailAddress]`
- `[MinLength(6)]`
- `[Range(1, 5)]`
- `[Range(1, 100)]`

Ez azért fontos, mert a rendszer már az API kérés feldolgozásakor képes kiszűrni a hibás vagy hiányos adatokat.

A client projekt

A `fitnessclassbookingwebapp.client` projekt az alkalmazás frontend része. Ez felelős a felhasználói felület megjelenítéséért, az oldalak közötti navigációért, az űrlapok kezeléséért, valamint az ASP.NET Core backend API végpontjaival történő kommunikációért.

A projekt Angular segítségével készült, TypeScript nyelven. Az Angular használata azért előnyös, mert komponensalapú felépítést biztosít, így az alkalmazás különböző részei jól elkülöníthetők egymástól.

Az Angular alapú client projekt hasznos, mert strukturált, bővíthető és jól karbantartható frontend alkalmazást eredményez. A komponensek külön kezelik az egyes oldalak működését, a service-ek külön kezelik az API kommunikációt, a modellek pedig típusosan írják le az adatokat.

A client projekt főbb feladatai:

- felhasználói felület megjelenítése
- navigáció kezelése
- bejelentkezés és regisztráció kezelése
- fitness csoportok listázása
- csoport részleteinek megjelenítése
- foglalások kezelése
- értékelések létrehozása és módosítása
- adminisztrátori oldalak megjelenítése
- backend API hívások végrehajtása
- felhasználói jogosultságok frontend oldali kezelése

AppRoutingModule

Az app-routing-module.ts fájl kezeli az alkalmazás útvonalait.

A projektben a következő fő útvonalak szerepelnek:

- /
- /login
- /register
- /groups
- /groups/:id
- /groups/:id/review
- /groups/:id/review/update
- /bookings
- /admin

A projektben route guard-ok is használatban vannak:

Az authGuard védi azokat az oldalakat, amelyeket csak bejelentkezett felhasználó érhet el. Ilyen például a foglalások oldala vagy az értékelés létrehozása. Az adminGuard biztosítja, hogy az admin felülethez csak admin jogosultságú felhasználó férhessen hozzá.

Angular Material

A projektben Angular Material komponensek is használatban vannak. Ezeket a material.module.ts fájl fogja össze. Az Angular Material előre elkészített, modern UI elemeket biztosít. Ez gyorsítja a fejlesztést, és egységesebb megjelenést ad az alkalmazásnak.

Például:

- a fitness csoportok megjeleníthetők kártyákon,
- az admin adatok táblázatokban listázhatók,
- az űrlapok Material input mezőket használhatnak,
- a sikeres műveletek snackbar vagy dialog formában jelezhetők,
- az admin felület sidenav alapú elrendezést használhat.

Service réteg a frontendben

Az Angular projektben külön service osztályok kezelik a backend API-val való kommunikációt. Ez jó megoldás, mert így a komponensek nem közvetlenül írják a HTTP hívásokat, hanem szolgáltatásokon keresztül kérik le vagy módosítják az adatokat.

AuthService

Az AuthService kezeli a bejelentkezést, regisztrációt, kijelentkezést és az aktuális felhasználói állapotot.

Főbb funkciói:

- regisztráció,
- bejelentkezés,
- kijelentkezés,
- aktuális felhasználó lekérése,
- token lekérése,
- bejelentkezett állapot ellenőrzése,
- szerepkör ellenőrzése,
- admin jogosultság ellenőrzése,
- e-mail cím létezésének ellenőrzése.

Frontend modellek

A client projektben külön TypeScript interface-ek írják le az alkalmazásban használt adatstruktúrákat. Ezek a modellek segítenek abban, hogy a frontend típusosan kezelje a backendtől érkező adatokat.

A Server projekt

A FitnessClassBookingWebApp.Server projekt az alkalmazás backend/API rétegét valósítja meg. Ez a projekt felelős azért, hogy az Angular frontend számára elérhetővé tegye az adatokat és az üzleti funkciókat HTTP API végpontokon keresztül.

A Server projekt központi feladatai:

- adatbázis-kapcsolat konfigurálása,
- dependency injection beállítása,
- JWT alapú autentikáció konfigurálása,
- jogosultságkezelés bekapcsolása,
- service réteg működtetése,
- üzleti logika megvalósítása,
- adatok kezelése,
- Angular alkalmazás kiszolgálása.

Ez a projekt kapcsolja össze a frontend működését az adatbázissal és az alkalmazás üzleti logikájával.

Program.cs

A Program.cs fájl az ASP.NET Core alkalmazás belépési pontja. Itt történik a backend működéséhez szükséges szolgáltatások regisztrálása és az alkalmazás middleware pipeline-jának beállítása. A Program.cs tehát az alkalmazás indítási és konfigurációs központja.

A fájlban konfigurálva van:

- az adatbázis-kapcsolat
- az Entity Framework Core
- a Unit of Work
- a service osztályok
- a JWT autentikáció
- az authorization
- a controller támogatás
- a HTTPS átirányítás

DbInitializer futtatása induláskor

Ez azt jelenti, hogy az alkalmazás induláskor ellenőrzi és előkészíti az adatbázist.

A DbInitializer feladata:

- migrációk alkalmazása

- kezdőadatok betöltése
- alap szerepkörök létrehozása
- tesztfelhasználók létrehozása

Service réteg

A Server projektben a fő üzleti logika service osztályokban található.

A service réteg azért fontos, mert a controller rétegnek nem közvetlenül kell az adatbázis-műveleteket és üzleti szabályokat kezelnie. A controller csak meghívja a megfelelő service metódust, a tényleges feldolgozás pedig a service-ben történik.

API és frontend kapcsolat

A Server projekt REST API-kat biztosít az Angular frontend számára. A frontend service-ek HTTP kéréseket küldenek a backend felé. A backend ezeket a kéréseket a controller és service rétegen keresztül feldolgozza, majd DTO-kat küld vissza válaszként.

Példák a kommunikációra:

- bejelentkezéskor a frontend elküldi az e-mail címet és jelszót
- a backend visszaadja a token és a felhasználói adatokat
- csoportlista megnyitásakor a frontend lekéri az összes csoportot
- foglaláskor a frontend elküldi az időpont azonosítóját
- a backend ellenőrzi a szabályokat, majd létrehozza a foglalást
- értékeléskor a backend ellenőrzi, hogy a felhasználó jogosult-e értékelni
-

Biztonsági megoldások

A Server projekt több biztonsági megoldást is tartalmaz.

Jelszó hash-elés

A jelszavak BCrypt segítségével hash-elve kerülnek eltárolásra. Ez azt jelenti, hogy az adatbázis nem tartalmazza a felhasználók eredeti jelszavát.

JWT token

A sikeres bejelentkezés után a rendszer JWT token generál. A token tartalmazza a felhasználó azonosítóját, e-mail címét és szerepköreit.

Role alapú működés

A tokenben szerepelnek a felhasználó szerepkörei, például Admin, Coach vagy Member.